

데이터 변환하고 요약하기



데이터 변환하기 : 변환의 필요성

데이터는 항상 원하는 타입으로 되어있지 않다!

	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계
0	2016-01-01	O	구름 조금	3,359	2,799	141	419	47	0	1,023	4,382
1	2016-01-02	O	구름 많음	5,173	4,370	203	600	100	111	2,092	7,265
2	2016-01-03	O	구름 많음	3,008	2,571	128	309	91	0	1,549	4,557
3	2016-01-04	X	구름 많음	890	602	NaN	235	51	223	800	1,690
4	2016-01-05	X	구름 많음	416	319	35	62	43	47	840	1,256

```
<class 'pandas.DataFrame'>
RangeIndex: 1086 entries, 0 to 1085
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   날짜        1086 non-null   str
1   공휴일      1086 non-null   str
2   날씨        946 non-null    str
3   유료합계    1086 non-null   str
4   어른       1086 non-null   str
5   청소년     1081 non-null   str
6   어린이     1086 non-null   str
7   외국인     1086 non-null   str
8   단체       1086 non-null   str
9   무료합계    1086 non-null   str
10  총계       1086 non-null   str
dtypes: str(11)
memory usage: 93.5 KB
```

- 실습에 사용할 데이터에 있는 숫자들이 정수 타입이 아닌 텍스트(str) 타입으로 저장되어 있음
- 동시에 천단위에서는 ,(콤마) 가 들어있음
- 텍스트(str) 타입일 경우 덧셈이나 뺄셈 등의 연산이 불가능

데이터 타입 변환 : `astype()`

해당 컬럼의 데이터 타입을 원하는 타입으로 변환할 때 사용

변환이 가능할 경우에만 변환이 이루어지고, 그렇지 않으면 에러가 발생

```
df["컬럼 이름"].astype(변환할 타입)
```

```
df["어른"] = df["어른"].astype(int)
```

df의 "어른" 컬럼 데이터의 타입을 int로 변환하여
df의 "어른" 컬럼에 저장(덮어쓰우기)



에러 발생

ValueError: invalid literal
for int() with base 10: '2,799'



데이터 값 대체 : replace()

.str.replace()

문자열 정제용

```
df['어른'] = df['어른'].str.replace(",", "", "")
```

- ",",를 ""로 바꿔 제거 → "2,799" → "2799"
- .astype(int)로 진짜 숫자(정수)로 변환
- .str 안 붙이면 에러 (문자열 표시)

df.replace()

값 자체를 치환

```
df.replace("X", 0)  
df.replace({"남": 1, "여": 2})  
df['공휴일'].replace("0", 1)
```

- 특정 값을 통째로 다른 값으로 변경
- 딕셔너리로 여러 값을 한 번에 치환

.str.replace() 는 문자열 컬럼 정제, df.replace() 는 값 자체를 다른 값으로 교체



데이터 값 대체 : replace()

	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계
0	2016-01-01	O	구름 조금	3,359	2,799	141	419	47	0	1,023	4,382
1	2016-01-02	O	구름 많음	5,173	4,370	203	600	100	111	2,092	7,265
2	2016-01-03	O	구름 많음	3,008	2,571	128	309	91	0	1,549	4,557
3	2016-01-04	X	구름 많음	890	602	NaN	235	51	223	800	1,690
4	2016-01-05	X	구름 많음	416	319	35	62	43	47	840	1,256
...	...	...	...	...	...	...	...	...	...	...	...
1081	2019-03-27	X	구름 많음	504	464	10	30	21	-	613	1,117
1082	2019-03-28	X	구름 많음	761	687	46	28	35	108	904	1,665
1083	2019-03-29	X	구름 조금	1,644	1,447	120	77	14	188	1,226	2,870
1084	2019-03-30	O	흐림	1,539	1,326	44	169	29	115	913	2,452
1085	2019-03-31	O	구름 조금	3,061	2,563	111	387	53	-	1,357	4,418

, 를 대체

-를 0으로 변경

데이터 타입 변환 : astype()

해당 컬럼의 데이터 타입을 원하는 타입으로 변환할 때 사용

	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계
0	2016-01-01	O	구름 조금	3359	2799	141	419	47	0	1023	4382
1	2016-01-02	O	구름 많음	5173	4370	203	600	100	111	2092	7265
2	2016-01-03	O	구름 많음	3008	2571	128	309	91	0	1549	4557
3	2016-01-04	X	구름 많음	890	602	NaN	235	51	223	800	1690
4	2016-01-05	X	구름 많음	416	319	35	62	43	47	840	1256
...	...	...	...	...	...	...	...	...	...	...	...
1081	2019-03-27	X	구름 많음	504	464	10	30	21	0	613	1117
1082	2019-03-28	X	구름 많음	761	687	46	28	35	108	904	1665
1083	2019-03-29	X	구름 조금	1644	1447	120	77	14	188	1226	2870
1084	2019-03-30	O	흐림	1539	1326	44	169	29	115	913	2452
1085	2019-03-31	O	구름 조금	3061	2563	111	387	53	0	1357	4418

```
df["어른"] = df["어른"].astype(int)
```

df의 "어른" 컬럼 데이터의 타입을 int로 변환하여
df의 "어른" 컬럼에 저장(덮어쓰우기)

```
<class 'pandas.DataFrame'>
RangeIndex: 1086 entries, 0 to 1085
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   날짜        1086 non-null   str
1   공휴일      1086 non-null   str
2   날씨        946 non-null    str
3   유료합계    1086 non-null   str
4   어른       1086 non-null   int64
5   청소년     1081 non-null   str
6   어린이     1086 non-null   str
7   외국인     1086 non-null   str
8   단체       1086 non-null   str
9   무료합계    1086 non-null   str
10  총계       1086 non-null   str
dtypes: int64(1), str(10)
```




데이터프레임 변환 시 유의사항

- Pandas의 메서드는 원본 데이터프레임(시리즈)을 바로 변환하지 않고, 변환된 데이터프레임(시리즈)을 반환
- 원본 데이터프레임을 변환하려면 덮어씌워주는 작업이 필요

```
df["어른"].astype(int)
```

df의 "어른" 컬럼 데이터의 타입을 int로 변환한
시리즈 그 자체

```
df["어른"] = df["어른"].astype(int)
```

df의 "어른" 을 astype으로 변환한 시리즈를 df의
"어른"컬럼에 덮어씌움으로써 원본 데이터프레임인
df가 변환됨

데이터 타입 변환 : to_numeric()

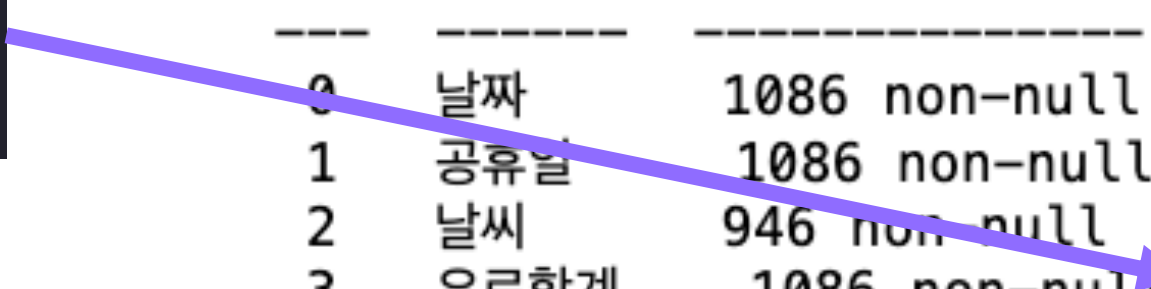
해당 컬럼의 데이터 타입을 숫자 타입으로 변환할 때 사용 (int형, float형 등..)

```
pd.to_numeric(df["컬럼이름"])
```

```
df["유료합계"] = pd.to_numeric(df["유료합계"])
```

df의 "유료합계" 컬럼의 데이터 타입을 숫자로 변환하여
df의 "유료합계" 컬럼에 저장(덮어쓰우기)

```
<class 'pandas.DataFrame'>  
RangeIndex: 1086 entries, 0 to 1085  
Data columns (total 11 columns):  
#   Column      Non-Null Count  Dtype  
---  -  
0   날짜        1086 non-null   str  
1   공휴일       1086 non-null   str  
2   날씨        946 non-null    str  
3   유료합계     1086 non-null   int64  
4   어른       1086 non-null   int64  
5   청소년     1081 non-null   str  
6   어린이     1086 non-null   str  
7   외국인     1086 non-null   str  
8   단체       1086 non-null   str  
9   무료합계    1086 non-null   str  
10  총계       1086 non-null   str  
dtypes: int64(2), str(9)
```





한꺼번에 타입 변환하기

변환해야 하는 컬럼들을 리스트로 만들고, for문을 활용해 한꺼번에 타입 변환 가능

```
columns=['유료합계', '어른', '청소년', '어린이', '외국인', '단체', '무료합계', '총계']  
for i in columns:  
    df[i]=pd.to_numeric(df[i])
```

```
<class 'pandas.DataFrame'>  
RangeIndex: 1086 entries, 0 to 1085  
Data columns (total 11 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   날짜        1086 non-null    str  
1   공휴일       1086 non-null    str  
2   날씨        946 non-null     str  
3   유료합계     1086 non-null    int64  
4   어른        1086 non-null    int64  
5   청소년      1081 non-null    float64  
6   어린이       1086 non-null    int64  
7   외국인       1086 non-null    str  
8   단체         1086 non-null    int64  
9   무료합계     1086 non-null    int64  
10  총계         1086 non-null    int64  
dtypes: float64(1), int64(6), str(4)
```

데이터 타입 변환 : to_datetime()

날짜/시간으로 변환 가능한 문자열, 정수, 실수를 시간 타입으로 변환할 때 사용

```
pd.to_datetime(df["컬럼이름"])
```

	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계
0	2016-01-01	O	구름 조금	3359	2799	141	419	47	0	1023	4382
1	2016-01-02	O	구름 많음	5173	4370	203	600	100	111	2092	7265
2	2016-01-03	O	구름 많음	3008	2571	128	309	91	0	1549	4557
3	2016-01-04	X	구름 많음	890	602	NaN	235	51	223	800	1690
4	2016-01-05	X	구름 많음	416	319	35	62	43	47	840	1256

```
df["날짜"] = pd.to_datetime(df["날짜"])
```

df의 "날짜" 컬럼의 텍스트(ex. "2016-01-01") 을
시간 타입(e.g. 2016년 1월 1일 시간 자료)으로 변환하여
df의 "날짜"컬럼에 저장(덮어쓰우기)

```
<class 'pandas.DataFrame'>
RangeIndex: 1086 entries, 0 to 1085
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   날짜        1086 non-null  datetime64[us]
1   공휴일      1086 non-null  str
2   날씨        946 non-null   str
3   유료합계    1086 non-null  int64
4   어른       1086 non-null  int64
5   청소년     1081 non-null  float64
6   어린이     1086 non-null  int64
7   외국인     1086 non-null  str
8   단체       1086 non-null  int64
9   무료합계    1086 non-null  int64
10  총계        1086 non-null  int64
dtypes: datetime64[us](1), float64(1), int64(6), str(3)
```



시간 타입을 활용한 데이터 변환 : dt

시간 타입은 날짜와 시간에 대한 정보가 담겨있어 다양한 처리가 가능

```
df['연']=df['날짜'].dt.year
```

dt.year를 사용하여 년도 정보가 저장된

df["연"] 컬럼을 생성

```
df['월']=df['날짜'].dt.month
```

```
df['일']=df['날짜'].dt.day
```

```
df['요일']=df['날짜'].dt.dayofweek
```

dt.dayofweek를 사용하여 요일 정보가 저장된

df["요일"] 컬럼을 생성(요일은 숫자로 저장 (월~일:0~6))

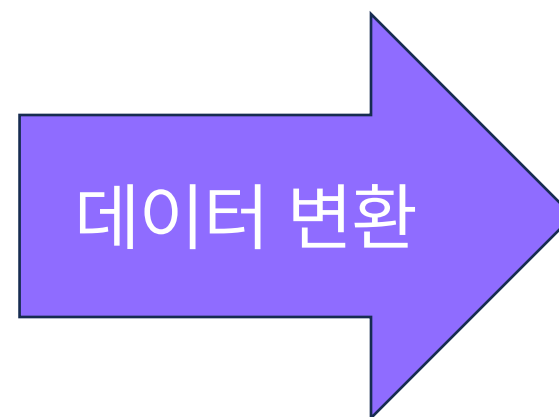
연	월	일	요일
2016	1	1	4
2016	1	2	5
2016	1	3	6
2016	1	4	0
2016	1	5	1



추가 데이터 변환

- 데이터 분석의 정확성을 높이기 위해 데이터의 값 변환이 필요한 경우가 많음
- 알아보기 쉬운 데이터로 바꾸거나, 단위를 통일하는 등 다양한 변환 가능

연	월	일	요일
2016	1	1	4
2016	1	2	5
2016	1	3	6
2016	1	4	0
2016	1	5	1



연	월	일	요일
2016	1	1	금
2016	1	2	토
2016	1	3	일
2016	1	4	월
2016	1	5	화



시리즈 연산을 통한 변환

- 데이터프레임의 각 열은 시리즈로, 연산이 가능함
- 다양한 컬럼의 연산을 활용해 데이터를 변환하거나 새로운 컬럼 생성 가능

```
df["어른"] = df["어른"] + 200
```

df의 "어른" 컬럼의 전체 값에 200을 더해 덮어쓰우기

```
df["동물원매출"] = df["어른"] * 5000 + df["청소년"] * 3000 + df["어린이"] * 2000
```

df의 "어른", "청소년", "어린이" 컬럼의 입장객 수에 해당하는 입장요금을 곱하고

전체를 더해 df에 "동물원매출" 컬럼 생성



시리즈 연산을 통한 변환

- 데이터프레임의 각 열은 시리즈로, 연산이 가능함
- 다양한 컬럼의 연산을 활용해 데이터를 변환하거나 새로운 컬럼 생성 가능

	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계	연	월	일	요일	동물원매출
0	2016-01-01	O	구름 조금	3359	3199	141.0	419	47	0	1023	4382	2016	1	1	4	15256000.0
1	2016-01-02	O	구름 많음	5173	4770	203.0	600	100	111	2092	7265	2016	1	2	5	23659000.0
2	2016-01-03	O	구름 많음	3008	2971	128.0	309	91	0	1549	4557	2016	1	3	6	13857000.0
3	2016-01-04	X	구름 많음	890	1002	NaN	235	51	223	800	1690	2016	1	4	0	NaN
4	2016-01-05	X	구름 많음	416	719	35.0	62	43	47	840	1256	2016	1	5	1	1824000.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1081	2019-03-27	X	구름 많음	504	864	10.0	30	21	0	613	1117	2019	3	27	2	2410000.0
1082	2019-03-28	X	구름 많음	761	1087	46.0	28	35	108	904	1665	2019	3	28	3	3629000.0
1083	2019-03-29	X	구름 조금	1644	1847	120.0	77	14	188	1226	2870	2019	3	29	4	7749000.0
1084	2019-03-30	O	흐림	1539	1726	44.0	169	29	115	913	2452	2019	3	30	5	7100000.0
1085	2019-03-31	O	구름 조금	3061	2963	111.0	387	53	0	1357	4418	2019	3	31	6	13922000.0

해당 변환들은 적용하지 않고 이후 이론 강의 진행



열 전체 변환 : map()

- 데이터프레임의 특정 열 전체를 변환하는데 사용

```
df["컬럼이름"].map(딕셔너리 등)
```

- 딕셔너리 등을 사용하여 변환 방식을 지정하고 해당 컬럼에 적용

데이터프레임에 map() 적용하기

df의 "요일" 컬럼의 값들을

딕셔너리 week의 Key와 Value를 활용하여

숫자로 표시되어 있는 요일을 글자로 변경

Key와 같은 값을

그 Key에 대응하는 Value로 치환

	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계	연	월	일	요일
0	2016-01-01	O	구름 조금	3359	2799	141.0	419	47	0	1023	4382	2016	1	1	4
1	2016-01-02	O	구름 많음	5173	4370	203.0	600	100	111	2092	7265	2016	1	2	5
2	2016-01-03	O	구름 많음	3008	2571	128.0	309	91	0	1549	4557	2016	1	3	6
3	2016-01-04	X	구름 많음	890	602	NaN	235	51	223	800	1690	2016	1	4	0
4	2016-01-05	X	구름 많음	416	319	35.0	62	43	47	840	1256	2016	1	5	1

```
week={0: '월', 1: '화', 2: '수', 3: '목', 4: '금',  
5: '토', 6: '일'}
```

```
df['요일']=df['요일'].map(week)
```

	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계	연	월	일	요일
0	2016-01-01	O	구름 조금	3359	2799	141.0	419	47	0	1023	4382	2016	1	1	금
1	2016-01-02	O	구름 많음	5173	4370	203.0	600	100	111	2092	7265	2016	1	2	토
2	2016-01-03	O	구름 많음	3008	2571	128.0	309	91	0	1549	4557	2016	1	3	일
3	2016-01-04	X	구름 많음	890	602	NaN	235	51	223	800	1690	2016	1	4	월
4	2016-01-05	X	구름 많음	416	319	35.0	62	43	47	840	1256	2016	1	5	화



데이터에 함수 적용: apply()

DataFrame에 함수를 적용할 때 사용
map과 달리 복수의 컬럼에 사용 가능

```
df["컬럼이름"].apply(함수, axis = 0 또는 1)
```

axis가 0이면 열 단위로, 1이면 행 단위로 함수 적용

index	국어	수학	영어
민수	95	88	96
도윤	72	72	68
서아	83	93	85
연우	89	80	94
하준	100	95	93

index	국어	수학	영어
민수	95	88	96
도윤	72	72	68
서아	83	93	85
연우	89	80	94
하준	100	95	93

☰ 데이터프레임에 apply() 적용하기

'눈'과 '비'를 '눈/비'로 합치는 weather함수를 선언

```
df["날씨"].value_counts()
```

날씨	count
구름 많음	277
구름 조금	236
맑음	222
비	101
흐림	100
눈	6
눈/비	4

Name: count, dtype: int64

```
def weather(e):  
    if e=='눈' or e=='비':  
        return '눈/비'  
    else:  
        return e  
  
df['날씨']=df['날씨'].apply(weather)
```

```
df["날씨"].value_counts()
```

날씨	count
구름 많음	277
구름 조금	236
맑음	222
눈/비	111
흐림	100

Name: count, dtype: int64

df의 "날씨" 컬럼의 값들에 weather함수를 적용하여 변환한 뒤
df의 "날씨" 컬럼에 저장(덮어씌우기)



lambda 함수를 활용한 apply 활용

*lambda 함수는 한 줄로 표현되며, : 왼쪽에는 인자를, 오른쪽에는 반환값을 작성하는 형식

```
def weather(e):  
    if e=='눈' or e=='비':  
        return '눈/비'  
    else:  
        return e  
  
df['날씨']=df['날씨'].apply(weather)
```



```
df['날씨']=df['날씨'].apply(lambda e : '눈/비' if e=='눈' or e=='비' else e)
```



데이터 요약하기

- 데이터의 정보를 요약해서 확인하기 위한 과정
- 데이터에 대한 정보를 미리 파악하여 추후 분석 방향 판단 가능
- 통계적 기법을 통해 도출한 데이터를 설명하는 다양한 통계 수치를 활용

데이터 통계값 확인 (mean, min, max, median)

```
df.mean()
```

```
유료합계    4595.651934
어른        3568.507366
청소년      463.038853
어린이      550.260589
외국인       75.399632
단체        892.831492
무료합계    2090.390424
총계        6686.042357
연          2017.177716
월           6.438306
일          15.670350
dtype: float64
```

```
df["컬럼이름"].mean()
```

```
df["어른"].mean()
```



```
3568.5073664825045
```

min(), max(), median() 등 다른 집계함수들도 같은 문법으로 활용 가능

데이터 통계값 확인 (mean, min, max, median)

평균

최솟값

최댓값

중앙값

```
df.mean()
```

```
df.min()
```

```
df.max()
```

```
df.median()
```

DataFrame 전체를 대상으로 하는 방법

```
df["컬럼 이름"].mean()
```

```
df["컬럼 이름"].min()
```

```
df["컬럼 이름"].max()
```

```
df["컬럼 이름"].median()
```

DataFrame의 특정 컬럼을 대상으로 하는 방법



데이터 전체 통계 : describe()

- 데이터프레임이나 시리즈의 간단한 통계 정보를 요약해서 확인 가능
- 평균(mean), 표준편차(std), 최소값(min), 25/50/75% 분위수(25%/50%/75%), 최대값(max) 출력

```
df.describe()
```

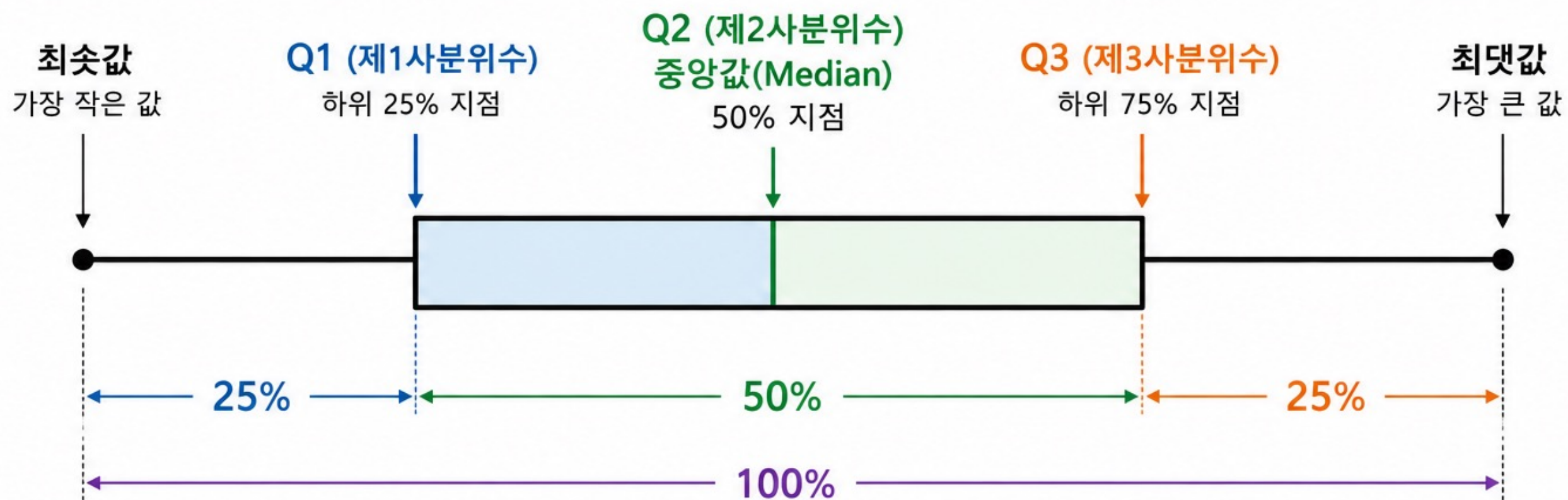
	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계	연	월	일
count	1086.000000	1086.000000	1081.000000	1086.000000	1086.000000	1086.000000	1086.000000	1086.000000	1086.000000	1086.000000	1086.000000
mean	4595.651934	3568.507366	463.038853	550.260589	75.399632	892.831492	2090.390424	6686.042357	2017.177716	6.438306	15.670350
std	6461.199603	5456.322438	1424.486882	1013.065306	103.030042	2155.583046	2299.475934	8469.956231	0.980313	3.415790	8.817452
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	2.000000	2.000000	2016.000000	1.000000	1.000000
25%	606.750000	462.500000	24.000000	43.000000	19.000000	0.000000	667.500000	1322.250000	2016.000000	3.000000	8.000000
50%	1697.000000	1282.000000	80.000000	150.000000	43.000000	73.000000	1182.000000	3030.000000	2017.000000	6.000000	16.000000
75%	5130.500000	3390.750000	298.000000	604.000000	87.000000	691.500000	2867.750000	8146.500000	2018.000000	9.000000	23.000000
max	40219.000000	38511.000000	17345.000000	20277.000000	1174.000000	18420.000000	22789.000000	58688.000000	2019.000000	12.000000	31.000000



통계 정보	의미
Count (개수)	결측값(NaN)을 제외한 데이터의 개수
Mean (평균)	데이터의 평균
Std (표준편차)	데이터가 평균에서 얼마나 퍼져 있는지
Min (최솟값)	데이터 중 가장 작은 값
Max (최댓값)	데이터 중 가장 큰 값



기술 통계량의 의미



통계 정보	의미
25% (1사분위수)	작은 값부터 정렬했을 때 25% 지점의 값
50% (중앙값)	정렬했을 때 한가운데 값 (= Median)
75% (3사분위수)	작은 값부터 정렬했을 때 75% 지점의 값



- 데이터 그룹화란 유사한 특성이나 값을 가진 데이터들을 동일한 그룹으로 분류하는 과정
- 데이터 그룹화의 장점
 - 대량의 데이터를 구조적이고 체계적으로 다룰 수 있음
 - 특정 그룹의 특성이나 동향을 쉽게 파악할 수 있음
 - 다양한 그룹 간의 비교를 통해 숨겨진 사실을 발견할 수 있음



실생활에서의 그룹화

000 총류 010 도서학, 서지학 020 문헌정보학 030 백과사전 040 강연집,수필집,연설문집 050 일반연속간행물 060 일반학회,단체,협회,기관 070 신문,언론,저널리즘 080 일반전집,총서 090 향토자료	100 철학 110 형이상학 120 인식론,인과론,인간학 130 철학의 체계 140 경 학 150 동양철학,사상 160 서양철학 170 논 리 학 180 심 리 학 190 윤리학,도덕철학	200 종교 210 비교종교 220 불 교 230 기 독 교 240 도 교 250 천 도 교 260 신 도 270 힌두교,브라만교 280 이슬람교(회교) 290 기타 제종교	300 사회과학 310 통 계 학 320 경 제 학 330 사회학,사회문제 340 정 치 학 350 행 정 학 360 법 학 370 교 육 학 380 풍속,예절,민속학 390 국방,군사학	400 자연과학 410 수 학 420 물 리 학 430 화 학 440 천 문학 450 지 학 460 광 물 학 470 생명과학 480 식 물 학 490 동 물 학
500 기술과학 510 의 학 520 농업,농학 530 공학,공업일반,토목공학,환경공학 540 건축공학 550 기계공학 560 전기공학,전자공학 570 화학공학 580 제 조 업 590 생활과학	600 예술 610 건 축 물 620 조각,조형예술 630 공예,장식미술 640 서 예 650 회화,도화 660 사진예술 670 음 악 680 공연예술,매체예술 690 오락,스포츠	700 언어 710 한 국 어 720 중 국 어 730 일본어,기타아시아제어 740 영 어 750 독 일 어 760 프랑스어 770 스페인어,포르투갈어 780 이탈리아어 790 기타제어	800 문학 810 한국문학 820 중국문학 830 일본문학,기타아시아문학 840 영미문학 850 독일문학 860 프랑스문학 870 스페인,포르투갈문학 880 이탈리아문학 890 기타제문학	900 역사 910 아 시 아 920 유 럽 930 아프리카 940 북아메리카 950 남아메리카 960 오세아니아 970 양극지방 980 지 리 990 전 기



도서관에서 읽는 책들은 '한국 십진분류법'에 따라 분류하여
 도서들을 주제에 따라 열 가지 유형으로 그룹화 해서
 구조적으로 책을 관리

마트에서는 유사한 유형의 상품들을 묶어 관리
 이를 통해서 소비자들이 편리하게 상품을 찾을 수 있음



데이터 그룹화: groupby()

```
df.groupby("컬럼이름1")["컬럼이름2"].집계함수
```

```
df.groupby("날씨")["총계"].mean()
```

df의 "날씨" 컬럼을 기준으로 그룹화하여
"총계" 값의 평균을 집계



```
날씨
구름 많음      6234.844765
구름 조금      7409.122881
눈/비
맑음           7756.225225
흐림           6056.450000
Name: 총계, dtype: float64
```

데이터를 그룹으로 묶어 분석할 때 사용

groupby()를 적용한 결과는 DataFrame 형태가 아닌 DataFrameGroupBy 형태



데이터 그룹화: groupby()

2개 이상의 컬럼을 기준으로 집계도 가능

```
df.groupby(["컬럼이름1", "컬럼이름2"])["컬럼이름3"].mean()
```

```
df.groupby(["날씨", "공휴일"])["총계"].mean()
```

df의 "날씨"와 "공휴일" 컬럼을 기준으로
그룹화하여 "총계"컬럼의 평균값을 집계

날씨	공휴일	
구름 많음	O	12994.262500
	X	3489.903553
구름 조금	O	14797.581081
	X	4034.148148
눈/비	O	4239.027778
	X	2462.400000
맑음	O	14658.260274
	X	4374.691275
흐림	O	9374.957447
	X	3113.622642



데이터 그룹화: groupby()

특정 컬럼을 지정하지 않고 전체 컬럼에 대한 집계도 가능

```
df.groupby("컬럼이름").집계함수
```

```
df.groupby("날씨").mean()
```

df의 "날씨" 컬럼을 기준으로
모든 컬럼의 평균값을 집계

	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총
날씨								
구름 많음	4278.678700	3362.703971	401.698182	506.801444	76.043321	819.425993	1956.166065	6234.8447
구름 조금	5119.974576	4012.063559	485.540426	587.555085	73.389831	883.622881	2289.148305	7409.1228
눈/비	2124.261261	1411.900901	282.765766	421.099099	47.720721	643.918919	914.342342	3038.6036
맑음	5271.693694	4051.324324	641.108597	575.518018	71.684685	1127.621622	2484.531532	7756.2252
흐림	4206.080000	3452.790000	208.650000	542.180000	78.810000	547.390000	1850.370000	6056.4500

데이터 그룹화: groupby()

여러 통계를 한 번에 확인하는 것도 가능

```
df.groupby("기준컬럼")["집계할컬럼"].agg(["집계함수1", "집계함수2", ...])
```

```
df.groupby("날씨")["총계"].agg(["mean", "sum", "count"])
```



	mean	sum	count
날씨			
구름 많음	6234.844765	1727052	277
구름 조금	7409.122881	1748553	236
눈/비	3038.603604	337285	111
맑음	7756.225225	1721882	222
흐림	6056.450000	605645	100